Querying a Codebase with NVIDIA Auto Ontology

Will Cook

4 November 2026

A case study of code-ontology-bridge

Abstract

NVIDIA Auto Ontology's database agent queried a relational map of its own Python source. The bridge extracts source facts and existing docstrings, then exports data and a semantic model through Auto Ontology's existing interfaces. A complete fresh run of 22 author-written questions produced 21 result tables matching the references. Twenty also had substantive explanations: one query selected an extra package, and a different answer returned correct rows with a one-word prose stub. The run used DeepSeek v4.1 Flash through OpenRouter, NVIDIA embeddings and reranking, explicit output requirements and fixed SQL routing. This is a development-set demonstration on one source snapshot; it does not measure an advantage over schema-only querying or direct source navigation.

1. From source navigation to database questions

A developer investigating a change needs to know what calls a function, which packages depend on it and where relevant tests might be. These questions combine descriptions with relationships spread across source files. A source map can store that information explicitly, allowing a database agent to turn a question into a query over the map.

The project connects two existing approaches. **NVIDIA Auto Ontology** associates relational tables and columns with descriptions, concepts, calculations and joins. Its agent retrieves context, generates SQL and executes it against connected data. **CodeMap**, an earlier source-navigation tool developed by Will Cook, supplies the starting idea of making saved descriptions and static connections available to people and coding agents. **code-ontology-bridge** adapts that idea into a standalone Python extractor and exports that Auto Ontology accepts. It reads docstrings rather than generating descriptions, and it does not require CodeMap's private interface.

The worked example asks Auto Ontology about its own implementation. The source snapshot contains 473 Python files, including 168 test files, and 5,319 symbols. A function is a row in the symbols table; a call relates two symbols. This makes structural code questions ordinary database questions.

2. The adapter

The extractor uses Python's AST to record files, symbols, class bases, calls and imports. It reads module and symbol docstrings, preserves source paths and line ranges, and records why a test file is associated with another file. The database has eight tables: files, headings, symbols, class bases, calls, imports, test links and a source snapshot. Two views summarize incoming calls and public symbols without test associations.

Three exports serve different purposes. A **DuckDB database** provides rows through Auto Ontology's existing connector. **Model YAML** describes tables, concepts, attributes and relationships through model import. An **evaluation dataset** supplies DDL, CSV data and reference questions in NVIDIA's evaluation format. Importing metadata alone does not connect the database. The

example ingests the database first, maps catalog identifiers, imports the model and compiles the semantic layer.

A separate compatibility patch configures public model clients and exposes Auto Ontology's existing `calculation_only` routing option at the chat boundary. This matters because a question about code can look like a metadata question, even though its answer is stored in the code-map rows. The patch also extends an existing placeholder validator. It adds no code-specific connector, SQL engine or query planner. Its twelve-file diff and scoped validation are retained with the patch documentation.

3. A caller chain, including a callback

Question 11 asks for every resolved caller symbol that reaches `resolve_semantic_fks`, directly or indirectly, with the shortest depth and excluding tests. The request explicitly includes standalone functions, methods and class-scope callers. The agent generated recursive SQL and returned:

Caller	Shortest depth
`_run_semantic_compilation`	1
`run_semantic_compilation`	2
`SemanticScheduler._run_once`	3
`main`	3

The scheduler passes the compilation function to `asyncio.to_thread`. A callback reference in the extracted call table connects it to the chain. The SQL follows reverse edges, avoids cycles and groups by caller identity to select the minimum depth. The full response preserves the request, intermediate events, SQL, complete symbol identifiers and prose.

The question includes a hint about reverse traversal and callbacks. No saved SQL example was imported into the model for this run. The answer describes callers in the recorded static map; a callback reference does not prove execution, and missing dynamic edges can hide other callers.

4. The fresh evaluation

All 22 questions were submitted once under protocol `explicit-output-v3`, with three concurrent requests and SQL routing enabled for every question. Auto Ontology's bounded internal retries remained enabled. There were no selected replacement answers. The run began at 11:27:10 UTC and completed at 11:50:35 UTC on 4 November 2026.

The chat model was `deepseek/deepseek-v4.1-flash-1017` through OpenRouter. Embeddings used `nvidia/nemotron-3-embed-1b`. Reranking used `nvidia/llama-nemotron-rerank-vl-1b-v2`. Eighteen questions contained authored hints. Every request specified required column names and types, with general guidance on distinct identities, caller kinds and numeric precision. Reference SQL and answer values were used for scoring after the response; they were not included in requests. Seven SQL examples remain available in the distributed dataset but were excluded from the imported model.

The scorer requires typed named columns, complete case-sensitive identities, correct row multiplicity and requested ordering. Integers match exactly; other numbers allow absolute transport error up to 1e-6. Successful HTTP completion is required. An automatic prose check rejects empty responses, known placeholders and generic unrelated refusals. It is supplemented by review of every final SQL statement, result table and explanation.

Measure	Result
Completed question requests	22 / 22
Tables matching all requested fields and rows	21 / 22
Matching tables with substantive explanations	20 / 22

Q2 asks what the semantic module does. The final query selects packages with `module LIKE '%semantic%'`, returning the intended `auto_ontology.semantic` package plus the server's semantic-compilation package. The extra row has an empty description; the reference selects only the first package. Q14 returns the correct seven resolution-statistics rows, but its prose is just "thought". The automatic score did not recognize that stub, so the separate review reduces the combined count from 21 to 20. Neither output has been edited or replaced.

Most other explanations describe the SQL and its assumptions, with the answer values carried by the accompanying table. The review does not certify polished standalone narrative answers. Matching rows on one database also does not establish query equivalence on other data: additional generated filters can leave this snapshot's rows unchanged.

5. What is and is not established

The demonstration establishes a working route from Python source to answers produced by Auto Ontology's database agent. Besides the caller chain, the agent found mutually calling packages, transitive import dependencies, and a base class with ten non-test subclasses.

The protocol was developed using these questions. Earlier full runs informed output requirements, identity guidance, caller wording and placeholder checks. Q11's wording was clarified to include all caller symbols; all 22 reference SQL statements and answer rows stayed unchanged. The result is therefore a development-set observation, with no independently authored held-out questions, second repository, schema-only control or comparison with an agent reading source directly. Earlier runs remain in Git history; the current result comes exclusively from the final complete run.

The underlying map has limits. Of 31,178 direct call sites, 9,442 have an internal callee identifier; 11,794 are classified as dynamic attribute calls without one. The small residual category named `unresolved` is not a measure of graph completeness. Another 1,046 module-level calls are omitted from the call table. Callback records are lower-confidence references, and test links describe imports or filename matches rather than executed coverage. Missing docstrings remain empty. Repeated qualified definitions, such as property getters and setters, receive distinct line-qualified identities, while ambiguous targets remain unresolved.

Queryable source representations are established techniques. CodeQL builds databases for source analysis; CodexGraph lets language-model agents query code graph databases. This project's contribution is the concrete adapter to Auto Ontology's contracts and its inspectable use on source-derived rows. Zhong, Yu and Klein (2020) evaluate SQL across test databases; the present single-snapshot row comparison has a narrower scope.

6. Additional commands and reproduction

The bridge also provides local commands for certification history, SQL-derived answer annotation and definition reconciliation. They record checks, plan updates to existing certification flags and associate returned SQL with candidate definitions. They do not enforce certified-only retrieval or trace which definitions the agent actually used. Their behavior and limits are documented separately in the governance guide.

The analyzed source is Auto Ontology revision `cdaf61e1bd0cf3820d11d72b43bb460816aebbd3`; the live runner used bridge revision `34285a2ed7440bc07bbe5e17f16452e6c48b50f0`. The run manifest records model settings and hashes of frozen inputs and runner files. Detailed results link every response, the unchanged automatic score, manual review and service captures. The reproduction guide separates local extraction, compatibility tests and a new credentialed live run.